

Post a Proposal Modal — Step-by-Step Coding Guide

1. Project Structure Path

- components/ProposalModal.jsx — main modal component.
- pages/dashboard.jsx — page containing the trigger button.
- styles/modal.css — modal styling.
- utils/validation.js — form validation logic.
- services/proposalService.js — API call setup (mock or real).

2. Step-by-Step Implementation

Step 1: Create a button (Post a Proposal) inside dashboard.jsx.

Step 2: Add useState to control modal visibility.

Step 3: Build ProposalModal component with four controlled inputs:

- title, modality, offeredSkill, neededSkill.

Step 4: Add validation rules: no empty fields, title ≥ 3 chars.

Step 5: Display errors inside the modal.

Step 6: On submit, run validation() from utils/validation.js.

Step 7: If valid, call proposalService.createProposal(data).

Step 8: For MVP use mock API and log results in console.

Step 9: Close modal after success.

Step 10: Apply CSS for responsive and clean UI.

3. Expected API Structure (Mock)

POST /api/proposals

{ title, modality, offeredSkill, neededSkill }

Response example:

{ status: 'success', id: 'uuid1234' }

4. What the Code Should Look Like (Flow Explanation)

- User clicks the button → modal opens.
- User fills inputs → useState updates values.
- User presses submit → validation runs.
- If validation fails → show error message.
- If validation passes → call API mock.
- Log data in console for MVP.
- Close modal and reset fields.

5. Recommended Technologies

- React useState, useEffect hooks.
- Axios or fetch for API calls.
- Simple CSS or Tailwind for layout.
- Optional: react-hot-toast for success messages.